

Rishivan

Rule Extraction & Retrieval Architecture

How classical Jyotish texts become executable, cited rules —
and how those rules produce an answer.

Prepared for	Lead Architect review & approval
Prepared by	Lokesh Kumar Saini · DianApps Development Team
Date	26 August 2026
Branch	<code>claude-ui-implement</code> @ 4134778
Engine state	1,503 rules loading · 2,000 tests passing
Status	Describes the system as built. Section 10 lists decisions requested.

Every figure in this document was measured against the working tree on the date above, not estimated. Where a number could not be re-verified at writing time (Qdrant and Postgres row counts, which need live services) it is marked last measured.

Contents

1. Purpose and scope	
2. System overview — two pipelines, one artefact	
3. Storage layers and what actually reads them	
4. Build-time pipeline	
5. Serve-time pipeline	
6. Architectural change this cycle — one rule system, not two	
7. Current state	verified
8. Model selection, cost and concurrency	
9. Quality controls and what they have caught	
10. Risks and decisions requested	needs sign-off
11. Roadmap	
12. Summary for the reviewer	

1. Purpose and scope

Rishivan answers questions about a birth chart using classical Jyotish texts, and grounds every statement it makes in a specific verse rather than in model recall.

One constraint shapes every decision in this document:

DESIGN INVARIANT

A claim that cannot be traced to a source verse must not be made. Not "should be cited where possible" — a claim without a resolvable citation is withheld, and the system says so rather than paraphrasing plausibly.

This document covers the system as it currently exists: the offline pipeline that turns scanned books into executable rules, and the request-time pipeline that turns a chart and a question into a cited answer. It is descriptive, not aspirational — where something is incomplete or unresolved, Section 10 says so plainly rather than presenting an intended end state as the current one.

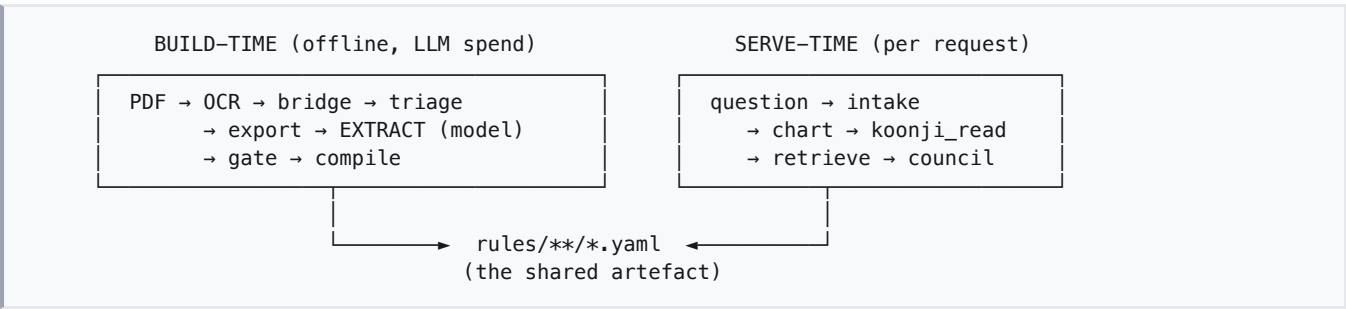
Out of scope: UI implementation detail, ephemeris and chart mathematics (treated here as a trusted dependency), deployment and infrastructure.

2. System overview — two pipelines, one artefact

The system is two pipelines that meet at a single artefact: a directory of compiled YAML rules. Keeping them separate is deliberate, and it is the reason serving is cheap and predictable.

	Build-time (offline)	Serve-time (per question)
Job	Books → executable rules	Chart + question → cited answer
Cost	LLM spend; once per book	Fixed; no extraction, ever
Reads	Postgres, JSONL on disk	YAML on disk, Qdrant
Trigger	Operator runs a CLI	User asks a question
Worst failure	A wrong rule reaches disk	A wrong answer, or a needless refusal

Rules are *compiled artefacts*. Nothing at request time calls a model to interpret a verse; that work is already done, checked, and frozen. The practical consequence is that answer latency and answer cost are independent of corpus size, and that a bad rule is a build-time defect we can find with tests rather than a runtime surprise.



3. Storage layers and what actually reads them

Four stores exist. Only two are on the request path, and that distinction has been a recurring source of confusion, so it is stated explicitly.

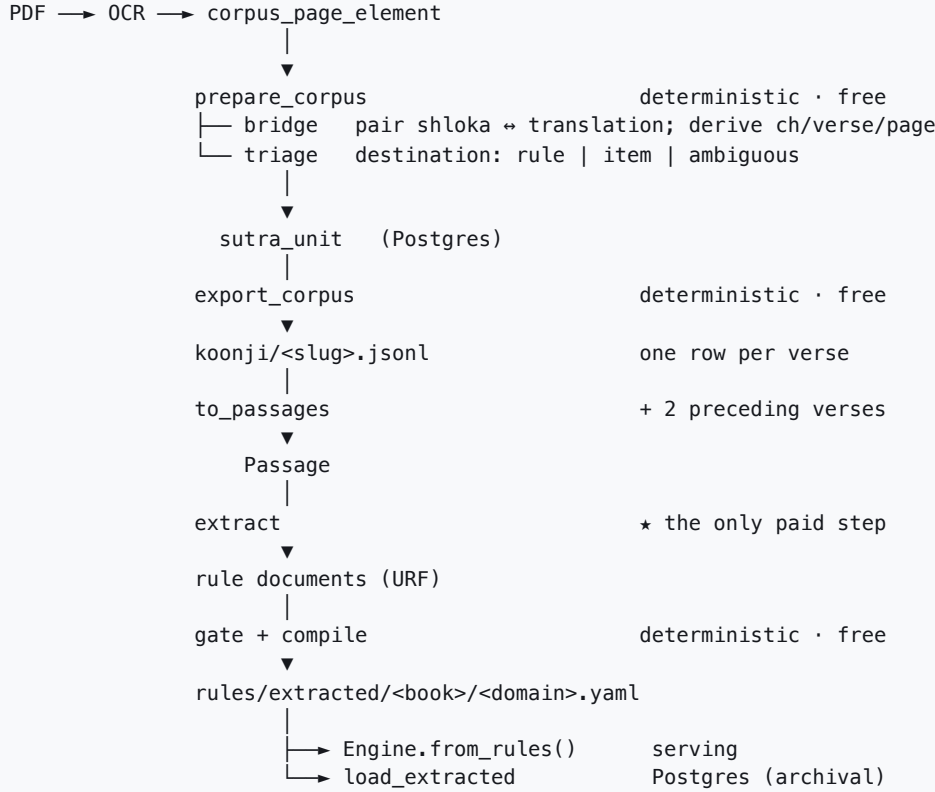
Store	Holds	Size	Read at serve time?
Postgres <code>document , corpus_page_element</code>	Uploaded books and their OCR text, page by page	23 books	No
Postgres <code>sutra_unit</code>	One verse each: shloka, translation, chapter, verse, page span	5,881	No
Postgres <code>rule</code>	Extracted rules, archival copy	1,515 *	No
Qdrant <code>rishivan_docs</code>	Passage vectors — the source pages shown to the user	52,958 *	Yes
Qdrant <code>rishivan_docs_rules</code>	Rule vectors, old format	1,046 *	No — superseded (§6)
<code>rishivan/koonji/rules/**/*.yaml</code>	Compiled rules — the live rule base	1,503	Yes

* last measured; requires live Qdrant / Postgres to re-verify. All other figures verified 26 Aug 2026.

FREQUENTLY MISREAD

Postgres is essential, but not on the request path. It is the corpus store — the extraction pipeline cannot run without it, because that is where bridged verses live. But no graph node queries it. The separation is enforced deliberately: `council/domains.py` duplicates a constant rather than importing the knowledge layer, specifically so SQLAlchemy stays off the Streamlit request path.

4. Build-time pipeline



4.1 Bridge and triage

Bridging pairs each Sanskrit shloka with its English translation and derives the locator — `chapter`, `verse_ref_local`, `page_from`, `page_to`. Each book carries an `EditionProfile`: data, not code, describing how that particular edition prints its chapter headings.

Triage then routes each unit to a destination — `rule` (states a rule), `item` (a definition, destined for the knowledge base), or `ambiguous`. Both steps are deterministic and free, and triage is the cost gate: it decides which units are worth spending model calls on.

Five books are refused, by design

Book	Why the bridge refuses it
vivaha-patalam	Entirely Devanagari; no Latin headings, no numbered divisions. 383 shlokas, zero English prose elements.
laghu-parashari	Sections are <i>named</i> , not numbered.
dharma-sindhu	No chapter headings survive in the OCR at all.
devakeralam vol 1 & 2	Prints "BOOK I (Containing 2718 Slokas)" — no chapter structure.

Filing these verses under an invented chapter number would fabricate the locator for an entire book, so the bridge skips rather than guesses. The cost is real and worth naming: *vivaha-patalam* is the marriage-timing text, and therefore the single most relevant source for the marriage question that originally exposed thin coverage.

The four numerology books are excluded for a different reason. The rule vocabulary is planet/sign/house-shaped and has no representation for a *mulank*; across all four books triage found 12 rule-bearing units. Extracting them would be spend with nowhere to land.

4.2 Extraction — and why it is now one call

Two modes exist and share all validation and compilation downstream.

Six-call mode (original): `classify` → `extract` ×2 → `reconcile` → `verify` → `back-translate`. Double extraction at differing temperatures surfaces disagreement; `verify` is adversarial, seeing the source and the JSON but not the extractor's reasoning; `back-translate` renders the JSON to prose *without* the verse, so divergence signals lost meaning.

Single-call mode (current): one call, extracting and classifying together. Adopted because **review is manual and external** — a verifier verdict the human reviewer will overrule is five calls spent pre-empting a judgement that was never the pipeline's to make. An empty `rules` array is the classification.

The switch was measured on the same 24 passages, not assumed:

Mode	Model calls	Rules produced
Six-call	58	2
Single-call	24	6

Fewer calls *and* more rules — the reconcile round was discarding rules the single pass keeps.

Everything free and deterministic is retained: verbatim-quote checking, `validate_candidate`, the full compiler pass sequence, and the review queue ordered by impact × uncertainty. What is lost is *recorded*, not implied: `verification_skipped` states why no verifier ran, and the generated YAML header repeats it, so a rule nothing examined can never be mistaken for one that passed a verifier.

4.3 The closed vocabulary

The model may only emit symbols the registry defines. This is the mechanism that makes extraction checkable rather than merely plausible.

Count	Kind	Examples
34	predicates	<code>occupies_bhava(subject, bhava)</code> , <code>varga_occupies(varga, subject, rashi)</code> , <code>dasha_active(system, subject, level)</code>
25	claims	<code>relationship.harmony</code> , <code>wealth.accumulation</code> , <code>career.disruption</code>
12	domains	career, education, health, longevity, progeny, property, relationship, spiritual, status, temperament, travel, wealth
6	schools	parashari, jaimini, prashna, lalkitab, tajika, muhurta
153	entity symbols	<code>graha.*</code> , <code>rashi.*</code> , <code>bhava.*</code> , <code>dignity.*</code> , <code>band.*</code> , <code>dist.*</code> , <code>ref.*</code> , <code>varga.*</code> , <code>nakshatra.*</code>

Three things are forbidden — approximating into the nearest predicate, dropping content, and inventing a symbol. One thing is required: where the vocabulary falls short the model must emit an **ExtensionProposal**. That is a correct and expected outcome, not a failure. The corpus is supposed to tell us what the vocabulary is missing.

WHY SCHOOLS ARE NAMESPACE, NOT MERGED

The traditions genuinely disagree. Jaimini uses rashi drishti where Parashari uses graha drishti — the same chart yields different aspects. Lal Kitab remedies contradict Parashari outright. Muhurta was given its own school this cycle for the same reason: a muhurta rule reasons about an *elected moment*, and filing it under Parashari would let it fire on a birth chart it was never about.

4.4 Identity is stamped by the caller, never authored by the model

Rule IDs and provenance are derived from the citation:

<code>{BOOK}. {TOPIC}. {LOCATOR}. X{nnnn}</code>	extracted	e.g. HINDUPREDICTIVE.LONGEVITY.CH14V5.X0001
<code>{BOOK}. {TOPIC}. {LOCATOR}. {nnnn}</code>	converted	(the X makes collision structurally impossible)

This was learned expensively, and it is worth the architect knowing both incidents because they characterise the failure class:

- **The engine stopped starting.** A model mistyped a verse number into an ID, two different Phaladeepika rules shared `PD.28.63.0002`, and the compiler refused the whole bundle — correctly, since a duplicate ID fires twice and counts as two independent sources under noisy-OR. One wrong digit in 274 rules took the application down.
- **One book appeared as three.** `source.book` used `setdefault`, so the model's guess won: `phaladeepika`, `Phaladeepika` and `phaladeepika-sastri-1950` all appeared. This one fails *silently* — an unrecognised book scores neutral in the source-authority matrix and drops out of weighting without ever raising.

Both are now overwritten from the passage, with mutation-verified regression tests — reverting to `setdefault` fails four of the five.

4.5 Gate and compile

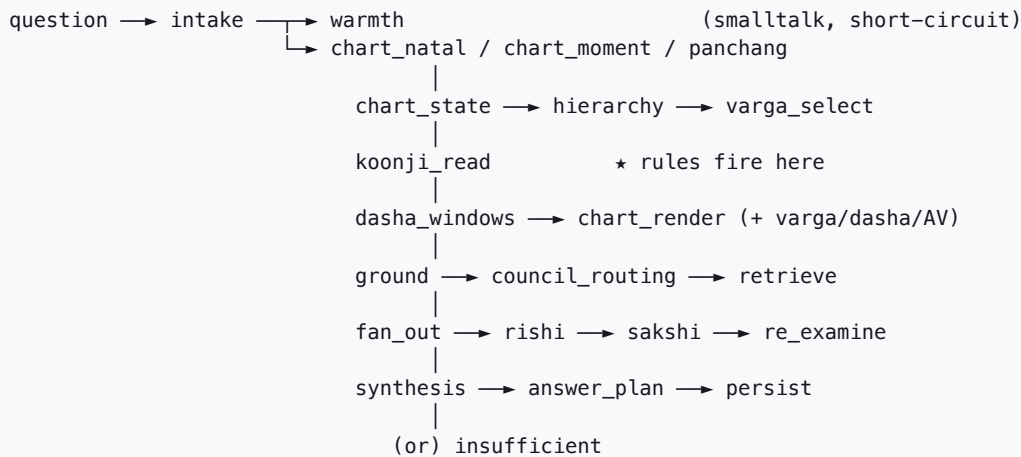
Candidate documents pass through the compiler: symbol resolution, typecheck, closure, DNF expansion, core extraction, contradiction and realizability checks, duplicate-ID and claim-polarity passes. Failures are dropped with their diagnostic attached; only survivors reach disk, and every one lands as `status: candidate`.

A LIMIT WORTH KNOWING BEFORE READING THE NUMBERS

A rule is expanded into every combination its `any` groups permit, capped at **32 variants**. Groups *multiply* when they sit side by side inside an `all`. One verse — "the quadrants are occupied by benefics" — expanded to four houses $\times$ four benefics = **1,536 variants** and was refused. The engine's own diagnostic now names the offending rule, so the gate drops it before it reaches disk instead of the failure surfacing at next startup.

5. Serve-time pipeline

A LangGraph state machine of 27 nodes. Topology lives in `NODE_NAMES`, `EDGE_MAPS` and `STATIC_EDGES` as module-level data rather than inline arguments, specifically so tests can walk it — a mistyped destination is otherwise a runtime `KeyError` on a branch nobody exercises until a user takes it.



5.1 Rule retrieval is not similarity search

This is the central design decision, and it inverts conventional RAG.

A conventional RAG asks "*which rules look like this question?*" Koonji asks "*which rules are **true** of this chart?*" — exact boolean evaluation against interned chart atoms. Similarity enters only afterwards, to rank which of the true rules to display.

MEASURED, NOT ASSUMED

Nominating by similarity first lost **11 to 14 of the 21 rules** genuinely true of a test chart. A similarity window cannot prefer what it has no way of knowing is true. Recall is therefore total by construction, and the failure mode conventional RAG has here — the rule is in the corpus, the embedding does not match, it never fires, and nobody finds out because you cannot compute recall against an unknown denominator — is eliminated rather than mitigated.

Mechanically, inside `Engine.read()`:

1. `all_chart_tokens(chart, when=query_time)` produces interned atoms. Dated by the *reading*, not the wall clock — matching *dasha* tokens against today while every other token came from a stated moment would evaluate a *Prashna* cast for the wrong time.
2. A `retrieval_plan` narrows candidates by status, school and domain.
3. The `RuleIndex` matches by **set containment** over an inverted index (`atom → variants requiring it`). There is not a single vector reference in `engine.py` or `index.py`.
4. Surviving rules fire, honouring exceptions.
5. Results enter an evidence graph: noisy-OR combination, plus **restatement clustering** so three paraphrases of one verse count as one piece of evidence rather than three.

The no-false-negatives invariant is maintained by *excluding* from the index core anything set membership cannot decide — negated leaves, numeric comparisons, unbound variables, and the undecidable. Each exclusion can

only widen the candidate set; the VM then prunes with exact arithmetic. False positives are free; a false negative would be invisible forever.

5.2 Passage retrieval — the other half, still Qdrant

Runs alongside rule matching and answers a different question: which pages should the user be shown. When a chart exists, `collect_chart_context` deliberately does *not* search the question. It embeds each of roughly 30 chart facts as its own query, ranks pages by how many distinct facts landed on them $\times$ source authority $\times$ the Book $\times$ Rishi matrix, takes the top 20, and widens each by ± 1 page so a shloka spanning a page break arrives intact. These become the "DRAWN FROM" chips.

DIVISION OF LABOUR

Pages come from Qdrant. Rules come from Koonji. Conflating the two is what produced a screen that could truthfully report "20 sources, 10 classical rules" while simultaneously refusing to answer — see §6.

5.3 Answer gating

`build_answer_plan()` decides what may be said. A claim must clear a 0.35 confidence floor. The plan is `insufficient` if nothing cleared it, or if every Rishi abstained. `sakshi_node` audits the plan before narration, and `re_examine` exists to send it back rather than let a weak plan through.

6. Architectural change this cycle — one rule system, not two

Before. The rules panel ("N classical rules match this chart") was filled by the older system reading Qdrant, while the Koonji engine ran separately and fed the written answer. The two held rules in incompatible formats and counted different corpora.

The consequence was worse than untidiness. Every rule extracted since the format changed — 274 at the time — fired correctly in the reading while being *invisible* in the panel. The counts on screen and the refusal underneath them were reporting on different indexes, so both could be true at once and the screen was still misleading.

After. `retrieve_node` builds the panel from the Koonji reading. `koonji_read` runs before it in `STATIC_EDGES`, so nothing is re-evaluated. Two new modules:

- `koonji/panel.py` — a reading, in the shape the panel reads.
- `koonji/describe.py` — a URF condition rendered in English, so the panel can state *why* a rule fired (`occupies_rashi(graha.moon, rashi.aries)` $\rightarrow$ "the Moon is in Aries").

The switch surfaced three real bugs, which is the argument for having made it: every rule was cited as "BPHS" (a hardcoded string, harmless while BPHS was the only sourced book); the panel came back silently empty because a bare attribute access raised inside the node's catch-all; and 39 rules carried no domain tag, making them uncitable by any Rishi.

7. Current state

Verified against the working tree, 26 August 2026.

7.1 Rule base — 1,503 rules loading

Directory	Rules	Provenance
rules/parashari/	8	Hand-written, human-reviewed
rules/converted/	1,109	Deterministic converter (BPHS bulk)
rules/extracted/	386	Single-call extraction, 5 books
Engine total	1,503	Loads clean; 2,000 tests passing

7.2 Extraction progress — 12 target books

Book	Passages	Rules	Model	State
phaladeepika	141	169	2.5-pro	Installed
hindu-predictive	304	112	3.5-flash-lite	Installed
brihat-jataka	13	98	2.5-pro	Installed
bhavartha-ratnakara	7	4	2.5-pro	Installed
prasna-marga	37	3	2.5-pro	Installed
muhurta-chintamani	412	56	3.5-flash-lite	Staged
prashna-tantra	339	—	—	Queued
saravali	557	—	—	Queued
jataka-parijata	687	—	—	Queued
sarvartha-chintamani	1,112	—	—	Queued

Corpus currently bridged and exported: **16 books · 5,587 units · 5,563 extractable passages**. Remaining spend to finish the queue is 2,695 calls, roughly **\$0.85** and about fifteen minutes (§8).

Rule-bearing density matters when reading yield. For *hindu-predictive*, triage marks only 74 of 304 passages as rule-bearing; the other 230 are definitions, worked examples or front matter. 112 rules from 74 rule-bearing passages is the number to judge, not 112 from 304.

7.3 Working tree

Nine files modified and one directory untracked, all part of this cycle's work and not yet committed:
koonji/prompts.py , koonji/index.py , koonji/pipeline.py , koonji/compiler.py , koonji/extract.py ,
koonji/__main__.py , scripts/extract_all.sh , tests/koonji/test_index.py , context.md , and
rules/extracted/hindu-predictive/ .

8. Model selection, cost and concurrency

Single-call mode uses the **deep model** only; `--fast-model` is inert, having served the classify and back-translate stages that single-call skips.

Model	Remaining 2,695 calls	Notes
gemini-2.5-pro	~\$114	Output is 91% of the bill
gemini-3.5-flash-lite	~\$0.85	Current choice — 23× cheaper

Measured on a real run: 304 calls in 33 seconds at 16 workers. The cost figures derive from measured token counts on completed books, not from list prices applied to estimates.

8.1 Prompt caching — now viable, worth doing later

Explicit context caching on flash-lite has a hard **4,096-token minimum**; implicit caching reported no hits at all. The system prompt was 2,641 tokens and did not qualify. After this cycle's prompt work it is **4,509 tokens and now qualifies** — reached by adding content that fixed real defects, not by padding to clear a threshold.

Enabling it requires client work (create the cache, pass `cached_content` , manage TTL) and would take input cost from roughly \$0.98 to \$0.31 across the remaining corpus. Real, but bounded by the input bill — it cannot compound, and it is not the constraint worth optimising first.

8.2 Concurrency required four thread-safety fixes

Three of these appear only under load, which is why they are listed: they are the class of defect a sequential proving run cannot find.

- **Client lazy-init race.** Two threads each built a Vertex client; the second assignment won, the first was garbage-collected and closed its transport mid-request. Symptom: *"client has been closed"* raised against perfectly good input, seven times in sixteen passages. Fixed with double-checked locking.
- **Budget.spend unlocked calls += 1.** The class exists to be a hard ceiling; an unlocked counter undercounts, defeating its only guarantee.
- **Bare JSON array.** The contract says "not a bare array" and models return one anyway. The six-call path normalised it in `reconcile` ; single-call lost whole passages until the parser accepted both shapes.
- **Bare `KeyError` for missing blocks and fields.** Now content errors that name the rule, the assertion kind and what was expected.

CORRECTION TO PRIOR REPORTING

Helicone was tagging every extraction call `gemini-2.5-flash` regardless of which model actually ran, because the dashboard property was stamped from `--fast-model` while the request used `--deep-model` . **All historical cost attribution in that dashboard is wrong.** Fixed. Authoritative sources are the CLI `budget:` line and GCP billing.

9. Quality controls and what they have caught

Control	Cost	Catches
Verbatim quote + SHA-256	Free	Fabricated text; silent post-hoc edits
Closed vocabulary	Free	Invented predicates, claims, domains
Stamped identity	Free	Duplicate IDs; book-name drift
Compiler passes	Free	Type errors, contradictions, unsatisfiable rules
Round-trip check	Free	Rules that cannot be re-read after writing
Variant limit (32)	Free	Index bloat; combinatorial blow-ups
Review queue	Free	Prioritises by impact × uncertainty
Manual review	Human	Everything above is necessary, none of it sufficient

9.1 The fabrication tripwire earns its strictness

The clearest catch: a passage consisting only of the chapter heading "*Certain self-evident Rajayogas*." produced a complete, confident Raja Yoga rule — ellipses and all — invented entirely from model knowledge. Blocked, because the quote did not appear in the passage.

9.2 Two extraction defects found and fixed this cycle

Both were found by inspecting output against source verses, not by any automated check — and that is the point: both produced rules that passed every automated control.

Dropped narrowing clauses

VERSE	"Even if the Moon is in the 8th no danger of Balarishta is indicated PROVIDED she occupies the Drekkanas of Mercury, Jupiter and Venus."
EXTRACTED	all[occupies_bhava(graха.moon, bhava.08)] ← proviso gone

The rule now asserts *protection* for exactly the charts the verse warns about. This is worse than producing nothing: it is well-formed, correctly cited, quote-verbatim, and passes every check. Only reading the logic against the verse reveals it is backwards.

Invented conditions — the mirror error

VERSE	"Sagittarius. – Mars is the best planet and in conjunction with Jupiter produces much good."
EXTRACTED	all[bhava_in_rashi(bhava.01, rashi.sagittarius), occupies_rashi(graха.mars, rashi.sagittarius), conjunct(graха.mars, graха.jupiter)] ← invented

An over-specified rule is not a cautious rule. It fires on fewer charts than the verse covers, so a reading silently loses evidence it was entitled to — and nothing in the output ever reveals that a rule failed to fire.

Both were fixed by adding a condition-completeness block to the prompt, covering both directions, and by teaching the non-obvious expansions the vocabulary requires (`varga_occupies` is indexed by *rashi*, so "the drekkanas of Mercury" must become the D3 signs Mercury owns). Also added: the value families for every argument kind, and the domain list — neither had ever been shown to the model, which is why it substituted `bhava.01` into a `distance` slot and inferred `domain.family` from the claim `family.father` .

Verified on held-out verses — not the prompt's own examples. Yield on *hindu-predictive* went from 47 to 112 rules over the same 304 passages, and the two defect classes did not recur.

10. Risks and decisions requested

10.1 Unreviewed rules are being served — decision needed

DECISION 1 · THE ONE I WOULD NOT SHIP PAST

`SERVED_STATUSES = {"candidate", "production"}`, and every rule in the engine is `candidate` — including the eight hand-written ones. Nothing is `production`, so that filter admits everything.

The approval gate lives in Postgres (`approved_at IS NOT NULL` , enforced by `MATCHABLE_PREDICATE`) and gates `embed_rules.py` → Qdrant. Since the §6 panel switch, nothing at serve time reads Postgres.

Approving a rule therefore has no effect, and neither does declining to.

To be precise about the history, because it is not a regression introduced carelessly: the *written answer* was never gated on approval — Koonji has always read YAML directly. The *panel* was gated, because it came from Qdrant and `embed_rules.py` publishes only approved rules. The panel switch removed the last place approval mattered.

Recommendation. Move the gate to where the rules live: approval flips YAML `status` from `candidate` to `production`, and serving accepts only `production`. The Postgres `rule` table then drops out cleanly and there is one rule system end to end.

The caveat that makes this a decision rather than a task. Flipping that switch today serves **8 rules instead of 1,503**, because nothing has ever been marked `production`. It needs a deliberate first pass — either bulk-promote the 1,109 converted BPHS rules (they came from the older reviewed pipeline), or accept `candidate` for now and record it as a known, signed-off gap.

10.2 One bad rule can still stop the engine — policy to re-affirm

DECISION 2

`Engine.from_rules()` calls `raise_for_errors()`, so a single unindexable or duplicate-ID rule prevents startup. This has happened twice — once from a duplicate ID, once from the 1,536-variant explosion.

Both root causes are now addressed (IDs stamped from the citation; index failures named per-rule so the gate drops them before they reach disk). But the underlying policy — *refuse to start rather than serve a corpus with a known error* — is a deliberate choice, and I would rather it were re-affirmed explicitly than left as an accident of where the check sits.

10.3 The legacy matcher cannot fire current-format rules

`satisfies()` returns `False` for an unrecognised atom type rather than raising — inert by design, *"not explosive"* as the code puts it. Rules loaded into Postgres and embedded to Qdrant therefore sit permanently silent there.

Harmless today, since Koonji feeds the answer. But it means the older system is dead weight rather than a fallback, and a fallback is what its presence implies. **Recommendation: retire it deliberately.** Maintaining two matchers by default is precisely how the counts diverged in §6.

10.4 Five books remain unreachable

They yield nothing and always will under the verse-unit model. **Page-level extraction is the only path.** Since citations are internal reference rather than user-facing prose, page granularity is acceptable — and the UI already displays page numbers.

Proposed as a **second, explicitly-labelled tier** (`citation_granularity: page`) so a page-cited rule can never be presented as equal in precision to a verse-cited one. Two trade-offs to accept going in: it loses the triage cost gate, since every page contains something; and it risks under-enumeration on dense pages, because a model given a batch tends to return a stable three to five items regardless of the true count.

10.5 Observability gaps

- **Helicone stores empty request and response bodies** for the Vertex passthrough. Its own error names the cause — *"Body has already been used"*: the gateway reads the stream to forward it and finds nothing left to log. Token counts and latency are accurate; bodies are not. Use `--record` to capture exchanges for inspection.
- **Helicone shows \$0.00** for flash-lite — no price entry for the model, so it computes zero.

10.6 Smaller items, recorded so they are not rediscovered

- Roughly 151 rules corpus-wide carry no domain tag, including 100 pre-existing BPHS rules. An untagged rule derives no Rishi affinity and can be cited by nobody — it is in the base and permanently silent.
- The bridge admits front matter and chapter headings as citable units — about 5% of spend, and *prasna-marga* is 68% table-of-contents. A deterministic filter is free and removes a fabrication vector, since a heading is exactly the input that makes a model invent what should follow.
- Classifier fields `needs_birth_data` and `is_followup` are computed on every request and read by nothing.
- The smalltalk decision is an unchecked single point of failure: misclassify a real question and the entire analytic path is skipped, returning a greeting. The deterministic router already scores 11 of 12 on this and could veto.

11. Roadmap

Immediate — completes the current corpus

1. Install staged *muhurta-chintamani* (56 rules); verify engine load.
2. Extract the remaining four books — 2,695 calls, ~\$0.85, ~15 minutes.
3. Commit this cycle's nine modified files.

Next — blocking for any external use

4. **Decide the approval-gate model (§10.1).**
5. Re-affirm or revise the fail-to-start policy (§10.2).
6. Retire or repair the legacy Qdrant matcher (§10.3).
7. Register missing domains; add the deterministic front-matter filter.

Later

8. Page-level tier for the five unbridgeable books (§10.4).
9. Explicit prompt caching — ~\$0.65 saving, needs client work (§8.1).

10. Rashi and graha alias table, built from observed extraction failures.
11. Retire the dead classifier fields.

12. Summary for the reviewer

What I believe is sound

Rule matching is exhaustive by construction rather than best-effort, and that choice was measured against the alternative rather than assumed. Provenance is stamped by the caller and never authored by a model, after two incidents that demonstrated why. Every generated artefact records what did *not* happen to it, so an unverified rule cannot be mistaken for a verified one. Extraction cost fell $23\times$ with no measured quality loss, and the two defects that did appear were found by held-out testing and fixed.

What needs your decision

The approval gate is disconnected: unreviewed rules reach users, and approval is a no-op in both directions. Everything else in Section 10 can be scheduled. That one I would not ship past, and the reason it is a decision rather than a fix is that closing it naively drops the served rule base from 1,503 to 8.

What is deferred deliberately

The legacy Qdrant matcher, the five unbridgeable books, and prompt caching — each with a recorded rationale rather than an assumption, and each listed above rather than left implicit.

VERIFICATION NOTE

Engine loads 1,503 rules. Test suite: 2,000 passed, 1 skipped. Both re-run immediately before this document was produced. Qdrant and Postgres counts are marked *last measured* in §3 because they require live services.